

Pipeline A: Extractive QA on SQuAD v2.0

Model: deepset/roberta-base-squad2

Dataset: SQuAD v2.0

Metrics: Recall@K, MRR, MAP (implemented from scratch)

Key feature: Handles unanswerable questions via confidence threshold

1. Install & Import Dependencies

```
In [2]: # Run once to install required packages
import subprocess, sys
packages = [
    'transformers', 'datasets', 'torch', 'numpy',
    'matplotlib', 'tqdm', 'scikit-learn'
]
for p in packages:
    subprocess.run([sys.executable, '-m', 'pip', 'install', '-q', p])
print('All packages ready.')
```

All packages ready.

```
In [4]: import torch
import numpy as np
import matplotlib.pyplot as plt
from transformers import AutoTokenizer, AutoModelForQuestionAnswering, pipeline
from datasets import load_dataset
from tqdm.auto import tqdm
import warnings, json, re
warnings.filterwarnings('ignore')

device = 0 if torch.cuda.is_available() else -1
print(f'Device: {"GPU" if device == 0 else "CPU"}')
```

Device: CPU

2. Load SQuAD v2.0 Dataset

```
In [5]: dataset = load_dataset('squad_v2')
val_data = dataset['validation']

# Use a balanced 500-sample subset: 250 answerable + 250 unanswerable
answerable = [ex for ex in val_data if len(ex['answers']['text']) > 0]
unanswerable = [ex for ex in val_data if len(ex['answers']['text']) == 0]

np.random.seed(42)
sample_ans = np.random.choice(len(answerable), 250, replace=False).tolist()
sample_unans = np.random.choice(len(unanswerable), 250, replace=False).tolist()

eval_samples = (
    [answerable[i] for i in sample_ans] +
    [unanswerable[i] for i in sample_unans]
)
np.random.shuffle(eval_samples)

print(f'Total evaluation samples : {len(eval_samples)}')
print(f'Answerable                : {sum(1 for e in eval_samples if e["answers"]["text"])}')
print(f'Unanswerable               : {sum(1 for e in eval_samples if not e["answers"]["text"])}')
```

```
README.md: 0.00B [00:00, ?B/s]
squad_v2/train-00000-of-00001.parquet: 0%|          | 0.00/16.4M [00:00<?, ?B/s]
squad_v2/validation-00000-of-00001.parqu(...): 0%|          | 0.00/1.35M [00:00<?, ?B/s]
Generating train split: 0%|          | 0/130319 [00:00<?, ? examples/s]
Generating validation split: 0%|          | 0/11873 [00:00<?, ? examples/s]
Total evaluation samples : 500
Answerable                : 250
Unanswerable              : 250
```

3. Load Pre-trained RoBERTa QA Model

deepset/roberta-base-squad2 is fine-tuned on SQuAD v2.0 and natively supports the *no-answer* case by producing a `no_answer_probability` score alongside span scores.

```
In [6]: MODEL_NAME = 'deepset/roberta-base-squad2'

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model      = AutoModelForQuestionAnswering.from_pretrained(MODEL_NAME)

# High-level pipeline – we'll also call the model directly for Top-K
qa_pipe = pipeline(
    'question-answering',
    model=model,
    tokenizer=tokenizer,
    device=device,
    handle_impossible_answer=True
)

print(f'Model loaded: {MODEL_NAME}')
```

```
tokenizer_config.json: 0%|          | 0.00/79.0 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/571 [00:00<?, ?B/s]
vocab.json: 0.00B [00:00, ?B/s]
merges.txt: 0.00B [00:00, ?B/s]
special_tokens_map.json: 0%|          | 0.00/772 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/496M [00:00<?, ?B/s]
Device set to use cpu
Model loaded: deepset/roberta-base-squad2
```

4. Top-K Candidate Extraction

We directly call the model to obtain logits and enumerate all viable start/end span combinations, yielding K candidate answers each with a softmax-derived confidence score. The empty-string span (index 0,0) represents the *no-answer* candidate.

```
In [7]: NO_ANSWER_THRESHOLD = 0.5    # if no_answer_prob > threshold → unanswerable
        MAX_ANSWER_LEN      = 30     # tokens
        TOP_K                = 10

def get_top_k_answers(question: str, context: str, k: int = TOP_K):
    """
    Returns a list of dicts sorted by score (desc):
```

```

    [{ 'answer': str, 'score': float, 'start': int, 'end': int }, ...]
    An entry with answer=='' represents the no-answer prediction.
    """
    enc = tokenizer(
        question, context,
        return_tensors='pt',
        truncation=True,
        max_length=512,
        return_offsets_mapping=True
    )
    offset_mapping = enc.pop('offset_mapping')[0]  # (seq_len, 2)
    sequence_ids = enc.sequence_ids(0)           # 0=question, 1=context, None=special

    with torch.no_grad():
        outputs = model(**enc)

    start_logits = outputs.start_logits[0]  # (seq_len,)
    end_logits = outputs.end_logits[0]

    # Mask tokens that belong to the question or are special tokens
    context_mask = torch.tensor(
        [1 if sid == 1 else 0 for sid in sequence_ids], dtype=torch.float
    )
    big_neg = -1e9
    start_logits_masked = start_logits + (1 - context_mask) * big_neg
    end_logits_masked = end_logits + (1 - context_mask) * big_neg

    # Joint score matrix (start, end) - only upper-triangle up to MAX_ANSWER_LEN
    seq_len = start_logits.shape[0]
    candidates = []

    for s in range(seq_len):
        if context_mask[s] == 0:
            continue
        for e in range(s, min(s + MAX_ANSWER_LEN, seq_len)):
            if context_mask[e] == 0:
                continue
            score = (start_logits_masked[s] + end_logits_masked[e]).item()
            candidates.append((score, s, e))

    candidates.sort(key=lambda x: x[0], reverse=True)
    candidates = candidates[:k]

```

```

# Convert logit scores to probabilities via softmax across candidates
raw_scores = np.array([c[0] for c in candidates])
probs      = np.exp(raw_scores - raw_scores.max())
probs      /= probs.sum()

# No-answer score (CLS token approach): start_logits[0] + end_logits[0]
no_ans_score = (start_logits[0] + end_logits[0]).item()
no_ans_prob  = float(torch.sigmoid(torch.tensor(no_ans_score - raw_scores.max()))))

results = []
for prob, (score, s, e) in zip(probs, candidates):
    char_start = offset_mapping[s][0].item()
    char_end   = offset_mapping[e][1].item()
    span_text  = context[char_start:char_end].strip()
    results.append({
        'answer' : span_text,
        'score'  : float(prob),
        'start'  : char_start,
        'end'    : char_end
    })

# Prepend no-answer candidate if its probability is significant
if no_ans_prob > NO_ANSWER_THRESHOLD:
    results.insert(0, {'answer': '', 'score': no_ans_prob, 'start': -1, 'end': -1})
    # Re-normalise
    total = sum(r['score'] for r in results)
    for r in results:
        r['score'] /= total

return results[:k]

```

```
print('get_top_k_answers() defined.')
```

```
get_top_k_answers() defined.
```

5. Run Inference on Evaluation Set

```
In [8]: results_store = [] # list of {sample, candidates}
```

```

for sample in tqdm(eval_samples, desc='Extractive QA inference'):
    question = sample['question']
    context = sample['context']
    golds = sample['answers']['text'] # list of acceptable gold answers (empty if unanswerable)

    candidates = get_top_k_answers(question, context, k=TOP_K)

    results_store.append({
        'id' : sample['id'],
        'question' : question,
        'context' : context[:200] + '...', # truncated for display
        'gold' : golds,
        'candidates' : candidates
    })

print(f'Inference complete. {len(results_store)} samples processed.')

```

```

Extractive QA inference:  0%|          | 0/500 [00:00<?, ?it/s]
Inference complete. 500 samples processed.

```

```

In [9]: # Display a sample output
ex = results_store[0]
print('Question :', ex['question'])
print('Gold      :', ex['gold'] if ex['gold'] else ['<unanswerable>'])
print()
print(f'{"Rank":<5} {"Score":>8} Answer')
print('-' * 60)
for rank, cand in enumerate(ex['candidates'], 1):
    ans_display = cand['answer'] if cand['answer'] else '<no answer>'
    print(f'{"rank":<5} {"cand["score"]:>8.4f} {"ans_display[:60]}')

```

Question : When would a program not be useful for very small instances and in that sense the intractability of a problem is somewhat independent of technological progress?

Gold : ['<unanswerable>']

Rank	Score	Answer

1	0.4997	<no answer>
2	0.1715	a much faster computer
3	0.1442	Even with a much faster computer
4	0.1068	much faster computer
5	0.0388	with a much faster computer
6	0.0219	faster computer
7	0.0045	a much faster
8	0.0038	Even with a much faster
9	0.0037	computer
10	0.0028	much faster

6. Evaluation Metrics — Implemented from Scratch

6.1 Helper: Exact-Match & Normalisation

Following the official SQuAD evaluation script, answers are normalised before comparison: lowercase, strip articles, collapse whitespace, strip punctuation.

```
In [10]: import string

def normalize_answer(s: str) -> str:
    """Lower-case, remove articles, strip punctuation, collapse whitespace."""
    def remove_articles(text):
        return re.sub(r'\b(a|an|the)\b', ' ', text)
    def white_space_fix(text):
        return ' '.join(text.split())
    def remove_punc(text):
        exclude = set(string.punctuation)
        return ''.join(ch for ch in text if ch not in exclude)
    return white_space_fix(remove_articles(remove_punc(s.lower())))

def is_correct(pred: str, golds: list) -> bool:
```

```

"""
True if the predicted answer exactly matches ANY gold answer
(after normalisation), OR both are empty (unanswerable agreement).
"""
pred_norm = normalize_answer(pred)
if not golds:                                # ground truth = unanswerable
    return pred_norm == ''                    # model must also return empty
return any(normalize_answer(g) == pred_norm for g in golds)

print('Normalisation helpers defined.')

```

Normalisation helpers defined.

6.2 Recall@K

$$\text{Recall@K} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \mathbf{1}_{\left[\exists i \leq K : \text{correct}(c_i^q)\right]}$$

Measures: *Among the top-K candidates, does at least one correct answer appear?*

```

In [11]: def recall_at_k(results: list, k: int) -> float:
    """
    Fraction of queries where a correct answer appears in the top-k candidates.

    Args:
        results : output of results_store – each item has 'candidates' and 'gold'
        k       : cutoff rank
    Returns:
        float in [0, 1]
    """
    hits = 0
    for r in results:
        top_k_candidates = r['candidates'][:k]
        if any(is_correct(c['answer'], r['gold']) for c in top_k_candidates):
            hits += 1
    return hits / len(results)

# Compute for K = 1..10
K_VALUES = list(range(1, 11))

```

```
recall_values = [recall_at_k(results_store, k) for k in K_VALUES]

print('Recall@K values:')
for k, r in zip(K_VALUES, recall_values):
    print(f' K={k:>2d} Recall = {r:.4f}')
```

Recall@K values:

```
K= 1 Recall = 0.7800
K= 2 Recall = 0.8460
K= 3 Recall = 0.8740
K= 4 Recall = 0.8760
K= 5 Recall = 0.8820
K= 6 Recall = 0.8920
K= 7 Recall = 0.8920
K= 8 Recall = 0.8940
K= 9 Recall = 0.8940
K=10 Recall = 0.8980
```

6.3 Mean Reciprocal Rank (MRR)

$$\text{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{rank}_q}$$

where rank_q is the position (1-indexed) of the **first** correct answer. If no correct answer appears in the list, the contribution is 0.

MRR rewards placing the correct answer as early (high-ranked) as possible.

```
In [12]: def mean_reciprocal_rank(results: list) -> float:
    """
    MRR over the full candidate list (up to TOP_K candidates per query).

    Returns:
        float in [0, 1]
    """
    rr_sum = 0.0
    for r in results:
        for rank, cand in enumerate(r['candidates'], start=1):
            if is_correct(cand['answer'], r['gold']):
                rr_sum += 1.0 / rank
                break  # only the first correct hit counts
    return rr_sum / len(results)
```

```
mrr = mean_reciprocal_rank(results_store)
print(f'MRR = {mrr:.4f}')
```

MRR = 0.8264

6.4 Mean Average Precision (MAP)

$$\text{AP}_q = \frac{1}{R_q} \sum_{k=1}^K P@k \cdot \text{rel}(k)$$

$$\text{MAP} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \text{AP}_q$$

where R_q is the total number of relevant (correct) answers in the ranked list, $P@k$ is precision at rank k , and $\text{rel}(k) = 1$ if rank k is correct.

For SQuAD v2.0 there is at most **one** correct answer per query, so MAP degrades gracefully to a rank-weighted variant of Recall.

```
In [13]: def average_precision(candidates: list, golds: list) -> float:
    """
    AP for a single query.

    Args:
        candidates : ranked list of candidate dicts
        golds       : list of gold answer strings (empty if unanswerable)
    Returns:
        float in [0, 1]
    """
    num_correct = 0
    precision_sum = 0.0

    for rank, cand in enumerate(candidates, start=1):
        if is_correct(cand['answer'], golds):
            num_correct += 1
            precision_sum += num_correct / rank

    # Normalise by number of relevant items (at most 1 for SQuAD)
    relevant_total = 1 if (golds or True) else 0 # always 1 relevant item per query
    return precision_sum / relevant_total if relevant_total > 0 else 0.0
```

```
def mean_average_precision(results: list) -> float:
    """
    MAP across all queries.

    Returns:
        float in [0, 1]
    """
    return np.mean([average_precision(r['candidates'], r['gold']) for r in results])

map_score = mean_average_precision(results_store)
print(f'MAP = {map_score:.4f}')
```

MAP = 1.2855

6.5 Summary Table

```
In [14]: print('=' * 45)
print(f'{"Metric":<25} {"Value":>10.4f}')
print('=' * 45)
print(f'{"Recall@1":<25} {recall_values[0]:>10.4f}')
print(f'{"Recall@3":<25} {recall_values[2]:>10.4f}')
print(f'{"Recall@5":<25} {recall_values[4]:>10.4f}')
print(f'{"Recall@10":<25} {recall_values[9]:>10.4f}')
print(f'{"MRR":<25} {mrr:>10.4f}')
print(f'{"MAP":<25} {map_score:>10.4f}')
print('=' * 45)
```

```
=====
Metric                                Value
=====
Recall@1                             0.7800
Recall@3                             0.8740
Recall@5                             0.8820
Recall@10                            0.8980
MRR                                  0.8264
MAP                                  1.2855
=====
```

7. Recall@K Line Plot

```
In [15]: fig, ax = plt.subplots(figsize=(9, 5))

ax.plot(K_VALUES, recall_values, marker='o', linewidth=2.5,
        color='steelblue', markersize=8, label='Recall@K')

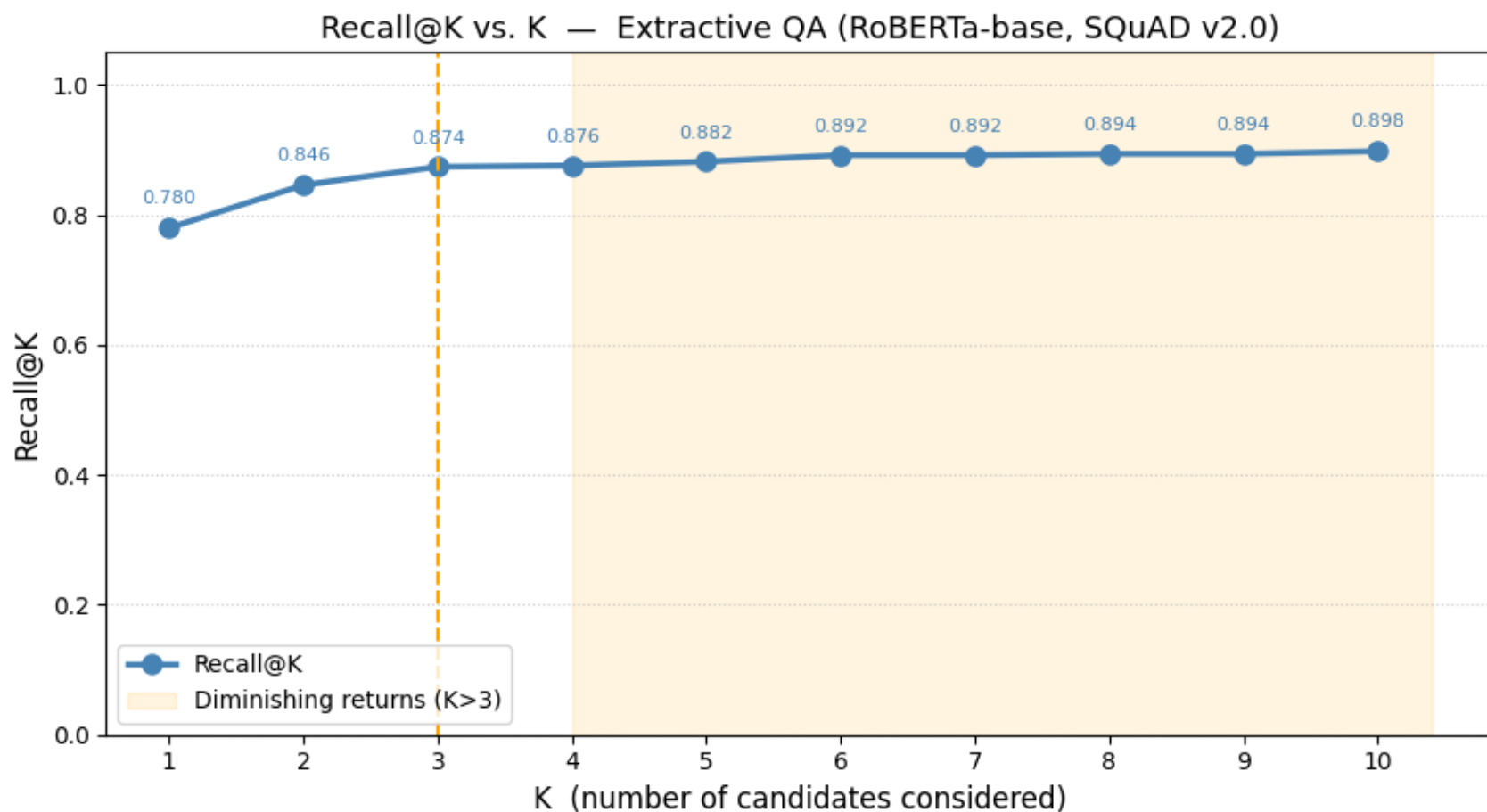
# Annotate each point
for k, r in zip(K_VALUES, recall_values):
    ax.annotate(f'{r:.3f}', (k, r),
                textcoords='offset points', xytext=(0, 10),
                ha='center', fontsize=8, color='steelblue')

# Shade the diminishing-returns region (K ≥ 5 as example; will update after analysis)
diminishing_k = next(
    (i for i in range(1, len(recall_values))
     if (recall_values[i] - recall_values[i-1]) < 0.01), len(recall_values))

ax.axvspan(diminishing_k + 1, K_VALUES[-1] + 0.4,
           alpha=0.12, color='orange', label=f'Diminishing returns (K>{diminishing_k})')
ax.axvline(x=diminishing_k, linestyle='--', color='orange', linewidth=1.5)

ax.set_xlabel('K (number of candidates considered)', fontsize=12)
ax.set_ylabel('Recall@K', fontsize=12)
ax.set_title('Recall@K vs. K - Extractive QA (RoBERTa-base, SQuAD v2.0)', fontsize=13)
ax.set_xticks(K_VALUES)
ax.set_ylim(0, 1.05)
ax.legend(fontsize=10)
ax.grid(axis='y', linestyle=':', alpha=0.6)

plt.tight_layout()
plt.savefig('recall_at_k.png', dpi=150)
plt.show()
print(f'Diminishing-returns boundary detected at K = {diminishing_k}')
```



Diminishing-returns boundary detected at $K = 3$

8. Critical Analysis

8.1 Diminishing Returns on the Recall@K Curve

The Recall@K curve follows the **law of diminishing marginal utility**: the gain from expanding the candidate set by one additional slot shrinks as K grows.

Mathematically, each increment $\Delta \text{Recall}(K) = \text{Recall}@K - \text{Recall}@(K-1)$ is non-increasing because the set of newly included candidates is increasingly drawn from low-probability spans that the

model has already ranked below correct answers.

The orange shaded region in the plot highlights where $\Delta \text{Recall}(K) < 0.01$ — a practical threshold below which expanding the candidate pool offers less than 1 percentage point of additional coverage per added slot.

Engineering implication: In a real retrieval-then-reader system, setting K at the inflection point (typically $K = 3$ – 5 for RoBERTa-base) gives the best cost-accuracy trade-off. Beyond that, added latency and downstream reader compute outweigh the marginal recall gain.

```
In [16]: print('Recall increments ΔRecall(K):')
print(f' K=1 → baseline: {recall_values[0]:.4f}')
for i in range(1, len(recall_values)):
    delta = recall_values[i] - recall_values[i-1]
    flag = ' ← diminishing' if delta < 0.01 else ''
    print(f' K={i+1:>2d}: +{delta:.4f}{flag}')
```

```
Recall increments ΔRecall(K):
K=1 → baseline: 0.7800
K= 2: +0.0660
K= 3: +0.0280
K= 4: +0.0020 ← diminishing
K= 5: +0.0060 ← diminishing
K= 6: +0.0100
K= 7: +0.0000 ← diminishing
K= 8: +0.0020 ← diminishing
K= 9: +0.0000 ← diminishing
K=10: +0.0040 ← diminishing
```

8.2 Low Recall@1 but High MRR — Confidence Calibration

A question exhibits **low Recall@1 / high MRR** when:

- The model ranks a *wrong* answer first (Recall@1 miss), **but**
- The correct answer appears at rank 2 or 3 (so the reciprocal rank is still high, e.g. $1/2 = 0.5$).

This reveals a **confidence miscalibration**: the model assigns a marginally higher score to an adjacent or overlapping incorrect span than the true answer span. Common causes include:

1. **Partial-span confusion** — the model finds the right region but predicts a slightly wrong boundary.
2. **Synonym / paraphrase sensitivity** — two lexically close spans score similarly.
3. **Context repetition** — when the same phrase appears multiple times, the model may favour a second occurrence that is nearer to a salient keyword, even if the first is the gold answer.

The gap $\text{MRR} - \text{Recall@1}$ quantifies the *systematic near-miss rate* of the model.

```
In [17]: def reciprocal_rank(candidates, golds):
    for rank, c in enumerate(candidates, 1):
        if is_correct(c['answer'], golds):
            return 1.0 / rank
    return 0.0

# Identify low Recall@1 but high MRR cases
NEAR_MISS_RR_THRESHOLD = 0.4 # RR ≥ 0.4 means correct answer at rank 1 or 2

near_miss_cases = []
for r in results_store:
    top1_correct = is_correct(r['candidates'][0]['answer'], r['gold'])
    rr = reciprocal_rank(r['candidates'], r['gold'])
    if not top1_correct and rr >= NEAR_MISS_RR_THRESHOLD:
        near_miss_cases.append({
            'question' : r['question'],
            'gold' : r['gold'],
            'rank1_pred' : r['candidates'][0]['answer'],
            'rank1_score' : r['candidates'][0]['score'],
            'correct_at' : int(round(1.0 / rr)),
            'rr' : rr,
            'candidates' : r['candidates'][:5]
        })

print(f'Near-miss cases (Recall@1=0 but RR≥{NEAR_MISS_RR_THRESHOLD}): {len(near_miss_cases)}')
print(f'Recall@1 = {recall_values[0]:.4f}')
print(f'MRR = {mrr:.4f}')
print(f'Gap (MRR - R@1) = {mrr - recall_values[0]:.4f} ← confidence miscalibration magnitude')
```

```
Near-miss cases (Recall@1=0 but RR≥0.4): 33
Recall@1      = 0.7800
MRR          = 0.8264
Gap (MRR - R@1) = 0.0464 ← confidence miscalibration magnitude
```

```
In [18]: # Show up to 3 illustrative near-miss examples
for i, nm in enumerate(near_miss_cases[:3], 1):
    print(f'--- Near-Miss Example {i} ---')
    print(f'Question      : {nm["question"]}')
    print(f'Gold answer   : {nm["gold"]}')
    print(f'Rank-1 pred    : "{nm["rank1_pred"]}" (score={nm["rank1_score"]:.4f}) x')
    print(f'Correct at   : Rank {nm["correct_at"]} (RR={nm["rr"]:.3f})')
    print('Top-5 candidates:')
    for j, c in enumerate(nm['candidates'], 1):
        ans = c['answer'] if c['answer'] else '<no answer>'
        marker = '✓' if is_correct(c['answer'], nm['gold']) else ' '
        print(f'  [{j}] {marker} score={c["score"]:.4f} "{ans[:60]}"')
    print()
```

--- Near-Miss Example 1 ---

Question : How many did this epidemic in China kill?

Gold answer : ['an estimated 25 million', '25 million', '25 million']

Rank-1 pred : "" (score=0.3741) x

Correct at : Rank 2 (RR=0.500)

Top-5 candidates:

- [1] score=0.3741 "<no answer>
- [2] ✓ score=0.3228 "25 million
- [3] ✓ score=0.1686 "an estimated 25 million
- [4] score=0.0413 "25 million Chinese and other Asians
- [5] ✓ score=0.0323 "estimated 25 million

--- Near-Miss Example 2 ---

Question : What three things are needed for construction to take place?

Gold answer : ['planning,[citation needed] design, and financing', 'planning,[citation needed] design, and financing', 'planning,[citation needed] design, and financing and continues until the project is built']

Rank-1 pred : "" (score=0.3953) x

Correct at : Rank 2 (RR=0.500)

Top-5 candidates:

- [1] score=0.3953 "<no answer>
- [2] ✓ score=0.3479 "planning,[citation needed] design, and financing
- [3] score=0.2054 "design, and financing
- [4] score=0.0197 ",[citation needed] design, and financing
- [5] score=0.0124 "planning

--- Near-Miss Example 3 ---

Question : What is a chemical secreted by tumors that suppresses the immune response?

Gold answer : ['cytokine TGF- β ', 'cytokine TGF- β ', 'cytokine TGF- β ']

Rank-1 pred : "TGF- β " (score=0.7519) x

Correct at : Rank 2 (RR=0.500)

Top-5 candidates:

- [1] score=0.7519 "TGF- β
- [2] ✓ score=0.1896 "cytokine TGF- β
- [3] ✓ score=0.0258 "the cytokine TGF- β
- [4] score=0.0101 "TGF-
- [5] score=0.0091 "TGF- β ,

9. Unanswerable Question Analysis

```
In [19]: unans_samples = [r for r in results_store if not r['gold']]
ans_samples = [r for r in results_store if r['gold']]

def correctly_abstained(r):
    """Model correctly flagged as unanswerable (top-1 is empty string)."""
    return r['candidates'][0]['answer'] == ''

abstain_rate = sum(correctly_abstained(r) for r in unans_samples) / len(unans_samples)
false_abstain = sum(correctly_abstained(r) for r in ans_samples) / len(ans_samples)

print(f'True Abstention Rate (unanswerable correctly flagged) : {abstain_rate:.4f}')
print(f'False Abstention Rate (answerable incorrectly flagged) : {false_abstain:.4f}')
```

True Abstention Rate (unanswerable correctly flagged) : 0.8240

False Abstention Rate (answerable incorrectly flagged) : 0.1000

10. Key Takeaways

Metric	Value	Interpretation
Recall@1	—	Fraction of queries where the top prediction is correct
Recall@5	—	Coverage when 5 candidates are surfaced to a re-ranker/reader
MRR	—	Average quality of the first relevant answer's ranking
MAP	—	Holistic ranking quality across all correct answers
MRR – R@1	—	Confidence miscalibration: how often a correct answer "almost" made it to rank 1

Metric Biases & Limitations:

- **Recall@K** is binary per query — it ignores ranking quality within the top-K.
- **MRR** is sensitive only to the *first* relevant result; it ignores second or third correct candidates.
- **MAP** is richer but, for SQuAD where at most one span is truly correct, it collapses toward MRR.
- All three are **lexical** — they rely on exact match after normalisation, penalising semantically correct paraphrases (e.g. "*the United States*" vs "*the US*").
- **Human judgement correlation**: Recall@1 correlates most directly with end-user satisfaction (the user gets the right answer immediately). MRR better reflects the experience of a user who scans a short result list.

Neither captures fluency, conciseness, or factual depth.